2026 EDITION

Network Security Laboratory

Lab 02 · Symmetric Cryptography & Steganography

Before We Start

The GNS3 lab from Lab 01 must be running. Verify connectivity before anything else.

Connectivity checks

- From Alice: ping the router (10.0.0.1)
- From Alice: ping Bob (10.0.0.20)
- From Alice: ping 8.8.8.8 (internet)

Install tools (once)

Alice & Bob netcat-openbsd · tshark · steghide · python3-pycryptodome

Darth tshark

tshark is the command-line version of Wireshark. We use it to inspect live network traffic from inside the VMs — this is how Darth eavesdrops.

Roles today

Alice	Bob	Darth
10.0.0.10 · sender / victim	10.0.0.20 · receiver / victim	10.0.0.30 · attacker / eavesdropper

If something doesn't work — raise your hand before moving on.

Today's Roadmap

1

OpenSSL — Encrypting Files & Messages

Encrypt a file with AES-256-CBC. Transfer it to Bob via netcat. Darth intercepts — and sees only ciphertext.

2

CryptoCat — Encrypted Chat Channel

Build a simple encrypted messaging tool. Observe how plaintext vs. encrypted traffic looks to an eavesdropper.

3

Steganography — Hiding a Message in an Image

Embed a secret message inside a JPEG. Send the image over the network. Extract the hidden text at the other end.

4

Python — Crypto with pycryptodome

Implement encryption in Python. Explore IV reuse attacks and why ECB mode is broken — hands-on with real code.

All exercises run inside Alice and Bob. Darth observes-

What is Symmetric Cryptography?

KEY

One key for both operations

The same secret key is used to encrypt the plaintext and to decrypt the ciphertext. There is no separate public key — just a single shared secret.

LOCK

The key is everything

The algorithm and the ciphertext can be public knowledge. Without the key, decrypting the message is computationally infeasible. The key is the only thing that must stay secret.

⚡

Fast — but requires a pre-shared key

Symmetric ciphers are extremely fast and efficient (hardware-accelerated AES on modern CPUs). The fundamental challenge: both parties must share the key before communicating. How?

Symmetric crypto: fast, secure, but requires a secure key exchange channel. That problem is solved in Session 03.

AES and Block Cipher Modes

AES-256 — Advanced Encryption Standard

Industry standard symmetric cipher. 256-bit key = 2^{256} possible keys. A key exhaustion attack on the fastest supercomputer would take longer than the age of the universe.

ECB

Electronic Codebook

Never use in practice.

Each 128-bit block encrypted independently. Same input block always produces the same output block — patterns in the plaintext are visible in the ciphertext.

CBC

Cipher Block Chaining

What we use in this session.

Each block is XOR'd with the previous ciphertext block before encryption. A random IV seeds the chain. Identical plaintext blocks produce different ciphertext — no pattern leakage.

GCM

Galois/Counter Mode

Best practice for new systems.

Counter mode + authentication tag. Encrypts and authenticates in one pass. Detects ciphertext tampering. Standard for TLS 1.3 and most modern protocols.

The Initialization Vector (IV)

A random value mixed into the first encryption block. Mandatory in CBC mode. Never reuse it.

With a fresh random IV	With a fixed or reused IV
✓ Same plaintext + same key = different ciphertext every time	✗ Same plaintext + same key = identical ciphertext every time
✓ An attacker cannot tell if Alice sent the same message twice	✗ Attacker sees that two messages are identical — without decrypting
✓ Patterns in the plaintext are hidden in the ciphertext	✗ In some modes (CTR) this is catastrophic: XOR of two ciphertexts reveals XOR of plaintexts
✓ The IV does not need to be secret — it is sent alongside the ciphertext	✗ In CBC it still leaks information about plaintext equality

Rule: always generate a fresh random IV for every encryption. Transmit it openly alongside the ciphertext — it is not a secret.

Exercise 1 — OpenSSL: Encrypting Files

openssl enc implements many standard symmetric ciphers directly from the command line. We use AES-256-CBC.

1a

Encrypt and decrypt a file

Write a message to a file. Encrypt it with AES-256-CBC using a shared password — the output is unreadable ciphertext. Decrypt it back and verify the original text is recovered.

1b

Try with the wrong key

Attempt to decrypt with an incorrect password. OpenSSL outputs garbage or an error — the ciphertext is unreadable without the exact key.

1c

Encrypt directly in the terminal

Pipe text through openssl without creating files. Copy the base64 ciphertext and decrypt it in the same terminal — end-to-end in one line.

1d

Transfer an encrypted file to Bob via netcat

Bob opens a listening socket. Alice sends the encrypted file over TCP. Bob decrypts and reads the original message. The key is never transmitted.

Exercise 1e — Darth Tries to Intercept

Darth captures the TCP session while Alice transfers the encrypted file to Bob.

Alice	Darth	Bob
Encrypts file Sends to Bob via TCP	Captures traffic Sees base64 ciphertext (cannot read content)	Receives file Decrypts with shared key Reads original message

What Darth sees in the captured traffic:

WITHOUT encryption (plain netcat)	*"Meet me at the library at 5pm"* ← *fully readable*
WITH AES-256-CBC encryption	*U2FsdGVkX1+8q7...* ← *base64 ciphertext, unreadable*

The algorithm and ciphertext can be public — without the key they are useless. Security lives in the key alone.

Exercise 2 — CryptoCat: Encrypted Chat

A Python wrapper around netcat that encrypts every message before sending and decrypts every message upon receipt. Both sides use the same key.

SCRIPT PROVIDED

2a

Plaintext baseline — Darth sees everything

Use plain netcat between Alice and Bob on port 9999. Darth captures the stream and reads every message in clear text. This is the before-picture.

2b

Run the encrypted channel

Open the provided cryptocat.py on Bob (server mode) and Alice (client mode), both with the same shared key. Type a message on Alice — Bob receives it decrypted. Darth still captures, but sees only base64 ciphertext.

2c

Try with the wrong key

Restart Bob's server with a different key. Alice connects with the original key. Bob receives garbage — the keys don't match. Communication is broken.

2d

How cryptocat.py works — read the code

Opens a TCP socket (server listens, client connects). For each message sent: encrypts with openssl enc, then transmits. For each message received: decrypts with openssl enc, then prints. Same AES-256-CBC from Exercise 1.

Both parties must share the key in advance. How do you exchange it securely? Asymmetric cryptography — Session 03.

Steganography vs. Encryption

Encryption

HIDES THE CONTENT

The message is transformed into unreadable ciphertext. An attacker knows that a secret message exists — they just cannot read it. Security depends on the key.

Steganography

HIDES THE EXISTENCE

The message is concealed inside an innocent-looking carrier (image, audio, video). An observer does not suspect that anything is hidden. Security through obscurity.

Combined approach

THE STRONGEST PROTECTION

Encrypt the message first, then hide the ciphertext inside an image. An attacker doesn't know a message exists — and even if they find it, they can't read it.

steghide uses the least-significant bits of image pixels — imperceptible to the human eye but detectable by specialised steganalysis tools.

Exercise 3 — Steganography with steghide

steghide embeds arbitrary data in the least-significant bits of JPEG pixel values. The image looks identical to the naked eye.

3a

Prepare the image and the secret message

Download a JPEG image (the cover file). Write a secret text message to a file. These are the two inputs to steghide.

3b

Embed the message inside the image

Run steghide embed: specify the cover image, the file to hide, the output image name, and a password. The output file looks identical to the cover — same image, tiny size difference.

3c

Send the image to Bob and extract

Transfer the steganographic image to Bob via netcat (same as Exercise 1). Bob runs steghide extract with the correct password and reads the hidden message.

3d

Try without the password

steghide info confirms that something is embedded (the statistical pattern is detectable), but it cannot reveal the content. Entering the wrong password fails extraction.

Steganography provides obscurity, not security. Combine it with encryption for real protection — hide the ciphertext, not the plaintext.

Exercise 4 — Python: pycryptodome

The command-line tools are convenient but limited. Real applications encrypt in code. pycryptodome is the standard Python crypto library.

SCRIPTS PROVIDED

1 Encrypt and decrypt in Python

Run the provided crypto_demo.py, experiment 1. A random 256-bit key and random IV are generated. The message is encrypted and then recovered. Run it twice — the ciphertext is different each time, even for the same message, because the key and IV are random.

Expected:
Key (hex), IV (hex), Plaintext, Ciphertext (changes each run), Recovered

2 IV reuse — what goes wrong

Run experiment 2. Alice encrypts the same message twice: once with a fixed IV, once with a random IV. With a fixed IV the two ciphertexts are identical — an attacker knows the messages are the same without decrypting anything.

Expected:
Fixed IV → Identical: True · Random IV → Identical: False

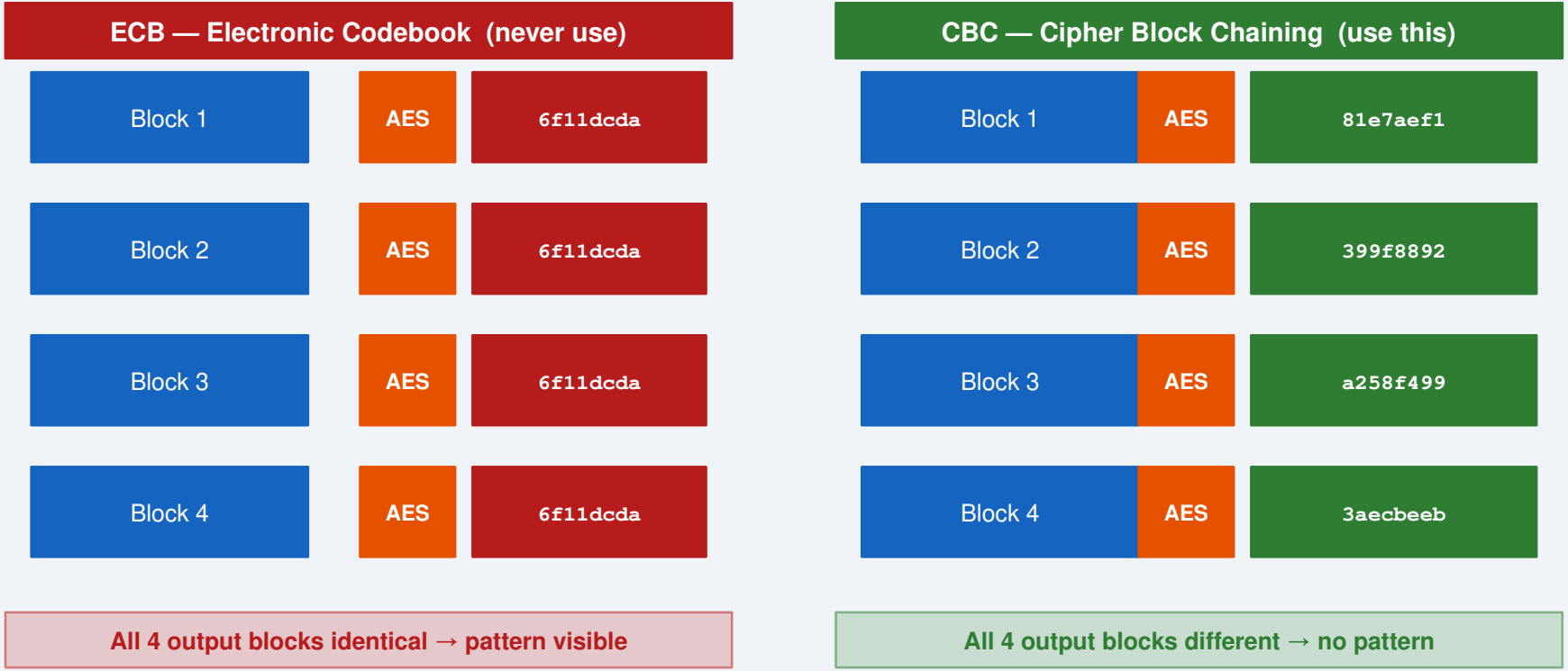
3 ECB mode — pattern leakage

Run experiment 3. A message made of four identical 16-byte blocks is encrypted in ECB mode and in CBC mode. ECB produces four identical output blocks — the structure of the plaintext is visible. CBC hides all patterns.

Expected:
ECB blocks: all identical · CBC blocks: all different

Why ECB is Broken — Block Mode Comparison

AES is a block cipher: it encrypts 128 bits at a time. The mode determines how consecutive blocks are linked.



ECB encrypts blocks independently. CBC XORs each block with the previous ciphertext — small change, massive security difference.

Verification Checklist

Run these checks before the session ends. All four must succeed.

1 OpenSSL file transfer

- ✓ Alice encrypts a file and sends it to Bob via netcat
- ✓ Bob decrypts and reads the original message
- ✓ Darth captures the stream — sees only ciphertext

2 CryptoCat encrypted channel

- ✓ Alice and Bob exchange messages through cryptocat.py with the same key
- ✓ Darth sees only base64 ciphertext in the captured stream
- ✓ Switching to a wrong key on one side breaks communication

3 Steganography

- ✓ Alice embeds a secret message in a JPEG and sends the image to Bob
- ✓ Bob extracts the hidden message with the correct password
- ✓ Entering the wrong password — extraction fails

4 Python pycryptodome

- ✓ All three crypto_demo.py experiments run without errors
- ✓ Experiment 2: fixed IV → Identical: True · random IV → Identical: False
- ✓ Experiment 3: ECB all blocks identical · CBC all blocks different

Common Problems & Fixes

tshark shows nothing

Verify you are capturing on the correct interface. On GNS3 VMs it is usually ens3 (Ubuntu) or eth0 (Raspbian). Run ip link show to list interfaces, then update the tshark command.

steghide embed fails: BMP or JPEG error

The downloaded image is not a real JPEG — some URLs serve PNG or WebP. Run the file command to check the format. If needed, install imagemagick and convert to a proper JPEG.

pycryptodome import error

The library is not installed or the wrong package is installed. Verify with a quick Python import. Install via apt (python3-pycryptodome) or pip3 if apt fails.

netcat connection refused

The receiver must start listening before the sender connects. Start Bob's nc command first, then run Alice's. Verify they are using the same port number.

cryptocat.py: decryption output is garbage

Both sides must use the exact same key string, including case and spacing. If the key contains spaces, quote it properly in the command. Also verify openssl is installed on both machines.

openssl: bad decrypt / wrong final block length

The key or the -base64 flag is inconsistent between encrypt and decrypt. Make sure you use exactly the same flags and key on both sides. Missing -base64 is the most common cause.

Key Takeaways

KEY Symmetric encryption is fast and strong — but requires a pre-shared secret key. Keeping the key secret is the entire security model.

IV Always use a fresh random IV for every encryption. Reusing the IV leaks information about the plaintext — even without breaking AES.

ECB Never use ECB mode. It leaks structural patterns from the plaintext. Use CBC for compatibility or AES-GCM for authenticated encryption.

STEG Steganography hides the existence of a message; encryption hides its content. Combining both gives stronger protection than either alone.

KEY? The key distribution problem: symmetric crypto cannot solve it on its own. Next session: public-key cryptography and how TLS handles key exchange.

What's Next

Session 03 · TLS/SSL & Public-Key Cryptography

PKI	How public-key cryptography solves the key distribution problem
TLS	TLS handshake: how HTTPS establishes a secure connection step by step
CERT	Certificates and Certificate Authorities — how trust is established on the web
MITM	Man-in-the-Middle against TLS — and why certificate validation matters